

目前、OS 设计概览

初始进程 PM

PM(Process Manager)是系统中的第一个用户态进程，它负责创建其他用户态进程，并为它们分配初始的权限与资源。pm 本身被赋予了最高权限(上帝模式)，它可以执行所有的特权系统调用，包括创建进程、分配内存、管理端口等。pm 的主要职责包括：

1. 负责引导完整用户态空间的建立，启动所有必要的系统服务器，分发对应的权限。
2. 管理系统中的进程，包括创建、终止进程。
3. T.B.D.

pm 的细节

pm 负责创建任务，这仅仅指的是创建一个新的 task 这样的调度个体，进程的程序映像的加载与虚拟内存的映射工作完全不在 pm 职责范畴之内。相反，这些工作将由专门的服务器来完成。可能是内核与它们沟通(pm 设置了初始 TCB 的字段，这样内核就知道找谁(pager))，可能是 pm 做了代理，代表请求方与其他服务器沟通。

在这个微内核架构的设计下，TCB 并非单一的。内核需要维护一份 TCB 列表，pm 同样需要维护一份，其它的什么服务器(比如某种监视服务器)出于需求也可能需要维护一份。但这些 TCB 并非完全相同，它们仅仅包含了各自所需的字段。

权限管理与 IPC

基本上，这分为两个方面。

粗粒度的 caps_t 标志位

一个无符号整数类型的字段 caps_t 将被放置在内核 TCB 中，用于标识任务的整体权限。每个权限对应了一组特定系统调用的访问权限。具体而言，这包括：

1. CAPS_GODMODE: PM(Process Manager, 首个被创建的任务)将被赋予的权限。允许执行一切系统调用。具备一个特殊系统调用的访问权：auth_delegate，即将自己的权限委托给其他任务。
2. CAPS_IRQ: 允许侦听和处理外设中断。对应的系统调用为：irq_listen 和 irq_unlisten。内核将在中断发生时通知注册了对应中断的任务。
3. CAPS_SYS: 允许进行进程相关管理。对应的系统调用为：task_spawn, task_kill。将来还会添加更多的进程管理相关系统调用。
4. CAPS_VM: 允许进行虚拟内存管理。对应的系统调用为：vm_map, vm_unmap, vm_grant(将本进程的某个页面映射的持有权转交给其他进程，也许映射位置或权限会发生变动)。
5. CAPS_PM: 允许进行物理页面的申请和管理。对应的系统调用为：pm_alloc(DMA 的需求), pm_acquire_at(MMIO 的需求)。
6. CAPS_COM: Communication。允许进行管理 Port 创建与转移相关的系统调用。对应的系统调用为：p_creat。
7. CAPS_DBG: 允许使用内核暴露的调试接口。对应的系统调用为：dbg_ps, dbg_printk(内核将维护一个环形缓冲区)等。
8. CAPS_NONE: 通常进程默认被赋予的权限。不允许进行任何特权系统调用。可以进行一些基本的系统调用，如：task_gettid, p_send, p_recv 等。

细粒度的 Port 机制

上文所提到的，Port 既充当了操作系统上 IPC 的核心机制，也发挥着权限管理的作用——通过限制进程对 Port 的创建权与收发权，辅以用户态服务器的端口管理策略，我们就能做到一定程度上安全且优雅的权限管理。

这个设计的来源是 Mach 系列微内核的端口机制。但做了一些简化与调整，以适应我们目前的需求。

先用一个例子走一遍可能的完整的流程：假设一个用户态进程 T 希望读取磁盘上某个文件。这将依次发生以下几个事件：

1. T 向 PNS(Port Name System)请求解析“fs”服务器对应的 Port。这里的 PNS 相当于 DNS 服务器，它的端口是我们预先设置好的（也不能称为硬编码，总之见下文）
2. pns 收到请求后，在自身维护的 Name-to-Port 映射表中查找“fs”对应的端口号，接着，它将创建一份 fs Port 的仅发送权限的副本，并将该仅发送端口所有权授予 T（即 T 获得了 fs Port 的发送权限）。注意，T 本身不具备创建端口的权利。
3. T 收到 pns 的回复后，便可以通过该 Port 向 fs 发送请求了。p_send 系统调用将在每个请求中携带一份属于 T 的临时端口的发送权，T 本身也会得到该临时端口的接收权，以便 fs 能够将结果返回给 T。这一临时端口将在请求完成后被销毁，对 T 本身是透明的。T 可能发送一系列请求：open, read, close.....等等。每发送一个消息，服务端都会获得这样一个临时端口的发送权，T 也同时获得该临时端口的接收权。通讯完成后，该临时端口将被销毁。
4. 某一时刻，T 完成了自己的操作，最后它将通过 p_close 系统调用关闭该 Port，回收资源。

接下来，我们再看看服务器端的工作流程，依旧以 fs 为例：

1. 初始进程 pm 在启动时，首先创建 pns 服务器，接着再开始创建各类传统的系统服务进程。fs 就将在此时被启动。
2. fs 启动后，首先会向 pns 注册自己对应的服务名“fs”，pns 会创建一个 Port，并将唯一的读权限授予 fs，以便它能够接收来自其他进程的请求。
3. 随后，fs 便会进入主循环，不断监听自己的 Port，每当收到请求时，便进行相应的处理。

端口通信的细节

Port 作为进程与系统服务之间沟通的桥梁，必须具备比网络中的 Socket 更严格的资源管理与控制机制。试想某个恶意进程大量创建端口，并向系统服务发送海量请求，这将极大地影响系统的稳定性与安全性。端口 ID 空间将被污染，整个系统的响应速度也将大幅下降。因此，我们不允许进程任意创建端口，而仅允许具备了 CAPS_COM 权限的进程创建和管理端口权限。pns 就是这样一个例子。

必须注意的一点是，此处的 Port 与计算机网络中的 Port 并不完全相同。不应该把它想象成某种 TCP 连接，而应该看作一种 mpsc 的单向消息队列。每个端口具备唯一的接收者，但可以同时存在零个或多个发送者。

为什么不效仿传统的双向 TCP 设计呢？比如服务器有一个固定的端口，进程有一个代表自身的端口，二者建立连接后，可以在连接上进行双向通信。每一个服务的请求者与服务器之间都存在一条独立的连接。这样设计的缺点在于，服务器需要维护大量的连接状态，这将极大地增加服务器的复杂度与资源消耗。此外，连接的建立与拆除也会带来额外的开销。而

且，这种设计下，服务器无法轻易地将请求转发给其他服务器，因为连接是绑定在特定的请求者与服务器之间的。相比之下，单向消息队列的设计更为简单高效。服务器只需监听一个端口，接收来自任意发送者的请求，并通过请求中携带的临时端口将结果返回给请求者。此外，服务器将轻而易举地具备了请求转发的能力，它只须将请求中的临时端口发送权一并转发给其他服务器即可！（更进一步地，我们可以创建 Proxy 服务器.....）

总之，再次重申，请你们把端口当成一种 mpvc 的单向消息队列。它并不固属于某个进程，而是可以在进程间转移的资源。

接下来看看有关的系统调用的初步设计吧。

`p_creat` 实际上会接收一个参数 `target`，这将指定希望创建的端口的号码。如果目标端口号已经被占用，则创建失败。此外，端口号将从 1 开始分配，如果指定的 `target` 为 0，则表明希望内核自动分配一个未被占用的端口号。

`p_transfer` 接收三个参数：`port`，`target_tid`，`mode`。它将把 `port` 的指定权限转移给目标任务。`mode` 是发送权限(`SEND_RIGHT`)、接收权限(`RECV_RIGHT`)和一些选项的按位或。有一个表示为 `DISCARD_RIGHT` 的选项，它将直接撤销该端口的指定权限。如果不指定目标任务的同时指定了该标志位，则表示希望销毁该端口的指定权限。如果指定了目标任务，则表示希望将该端口的指定权限转移给目标任务。注意，`SEND_RIGHT` 默认在被转移时不会从当前任务撤销，而如果就算不指定 `DISCARD_RIGHT`，`RECV_RIGHT` 也会被撤销。所谓 mpvc。

调用 `p_transfer` 的进程必须具备希望转移的权限，否则会被判为非法。`pns` 服务器在服务注册后会把端口接收权转交给服务器，自身会保留一份发送权，这样它就能在进程进行端口解析时将发送权复制一份交给请求者。

另一个有意思的点是，端口号本身是被定义为一个无符号整型数字，不过就算进程知道了某个端口号，它也不一定就能使用该端口。因为进程必须持有该端口的发送权才能向该端口发送消息。同理，进程必须持有该端口的接收权才能从该端口接收消息。因此，端口号本身并不构成访问控制的依据，仅仅是一个标识符而已。

最后一个问题：进程是怎么和 `pns` 服务器取得联系的呢？答案是，`pm` 自身保留了一份 `pns` 服务器的端口发送权。每个进程被创建时，`pm` 都会把这份发送权的副本交给新进程。这是理所当然的，正如我们应该预先知道 DNS 服务器的 IP 地址一样。

Pager

Pager 是另一个重要的用户态服务器，用来管理进程的虚拟内存空间，同时负责处理缺页异常，按需加载页面。

内核 TCB 中存在一个字段 `pager`，它指向该任务的 `pager` 进程 id。`pm` 创建完新任务后，并不负责任何虚拟内存的映射工作。它仅仅指定了该任务的 `pager`。当任务运行过程中发生缺页异常时，内核将暂停该任务的执行，并向其 `pager` 发送一条缺页异常的消息，消息中包含了缺页的虚拟地址等信息。`pager` 收到该消息后，便可以根据自己的策略决定如何处理该缺页异常。它可能选择从磁盘加载相应的数据，或者分配一个新的物理页面，然后通过 `vm_map` 系统调用将该页面映射到任务的虚拟地址空间中。处理完毕后，`pager` 将向内核发送一条完成消息，内核便会恢复任务的执行。

为何特地设置一个字段呢。因为这样一来，我们可以为不同的进程指定不同的 `pager`，从而实现更为灵活的内存管理策略。例如，大部分进程可能仅仅使用默认策略的 `pager`，而一些特殊进程则可能使用自定义的 `pager` 来实现特定的内存管理需求。此外，我们准许多个同类 `pager` 并存，这就漂亮地实现了负载均衡！

USpace 现状与一些构想

目前，用户空间的设计还处于初步阶段，许多细节尚未完全敲定。但我们已经确定了一些核心的、最基本的设计与服务。

启动流程

1. 内核启动后做一些硬件和资源的初始化，然后创建第一个用户态进程 pm，它的 elf 文件被嵌入了内核映像。内核会为它创建完整的虚拟空间映射，并把它完全加载到内存中。接着，内核开始调度。
2. pm 启动！此时，它处于上帝模式，具备一切权限。我们在构建 pm 映像的时候，会同时打包一系列必要的服务器 elf 文件。支持一个最小的多任务操作系统的服务器有 pns, pager, uart 和 shell。在此之上，我们可以指定更多的服务器进程，比如 fs, net 等等。

pm 启动后，首先启动一个默认的 pager，为它分发 CAPS_VM 与 CAPS_PM 权限。至此就可以不必再做任何内存映射相关的工作了。接着，pm 启动 pns 服务器，为它分发 CAPS_COM 权限。至此 IPC 机制就绪。再后，pm 可以启动更多的系统服务进程，并分发对应所需权限。最后，pm 启动 shell。它不需要任何特权，换句话说，CAPS_NONE。

USpace 的服务器们

下面是一些必须被实现的服务器：

1. pm
2. pns
3. pager 提供一个默认的全局 pager。
4. uart driver
5. virtio-block driver
6. fs

下面是一些可以考虑实现的服务器：

1. virtio-net driver
2. net
3. kdb kernel debugger。具备 CAPS_DBG 权限。作为内核调试的前端。
4. 'monitor's。 具备 CAPS_DBG 权限。各式各样的监视服务器，用来监测或捕获系统中的特定事件。
5. 'proxy's 某些服务的代理服务器。通过代理服务器，我们可以实现负载均衡，以及某些特殊的权限管理策略。

How to 编程 in USpace?

用户态程序被划分为两类。App 与 Server。App 是传统意义上的应用程序，它们通过与各种服务器通信来完成自己的任务。Server 则是提供某种服务的进程，它们监听特定的端口，并响应来自 App 的请求。它们都可以完全在用户态完成，而不必修改内核。

1. App 可以通过 pns 解析各种服务器的端口，并通过端口与它们通信。也可以更简单地依赖于某种高级封装的 IPC 库来完成这些工作，使得这些操作看起来与宏内核下的系统调用无异。我们将在用户库提供一些这样的封装。

2. Server 需要监听特定的端口，并在主循环中不断接收请求并进行处理。目前，尚需要修改 pm 代码来启动这些服务器，分发对应权限，并将其 elf 映像打包进 pm 映像。服务器需要定义好自己的协议，以便与 App 进行通信。我们要求“消息”这一实体的大小不超过一定限制，同时靠前一些的字节必须用于标识。定义好的协议应该作为头文件放在 include/ospace/servers 目录下，以便 App 与 Server 双方包含使用。

悬而未决的问题们

- pm 分发完权限之后，要不要把自己的 CAPS_GODMODE 撤销掉？如果撤销掉，某个服务器出现问题时，pm 如何再启动并分发权限给它？如果不撤销掉，pm 本身就成了一个单点故障，这与微内核设计的初衷相悖。
- 消息协议的遵循并非强制性的。我们没办法做到类型安全，这不太优雅。此外出于简单考虑，我们没有引入 IDL(Interface Definition Language)，这是合适的吗？而且，我们并不能做到对消息结构体的格式的强制约束，这不太完善。最后，我们现在的做法(头文件共享)并不适合跨语言编程，作为一个 OS 而言未免有点尴尬。